

APDS7311 PoE 1 breakdown

Solutions Architecture

Remember to refer to the marking guidelines and reference your work accordingly. All work must show **an in-depth explanation that shows evidence of additional research and effort is provided.**

Reference all work using [Harvard - Anglia Style Reference Guide](#).

MobSF

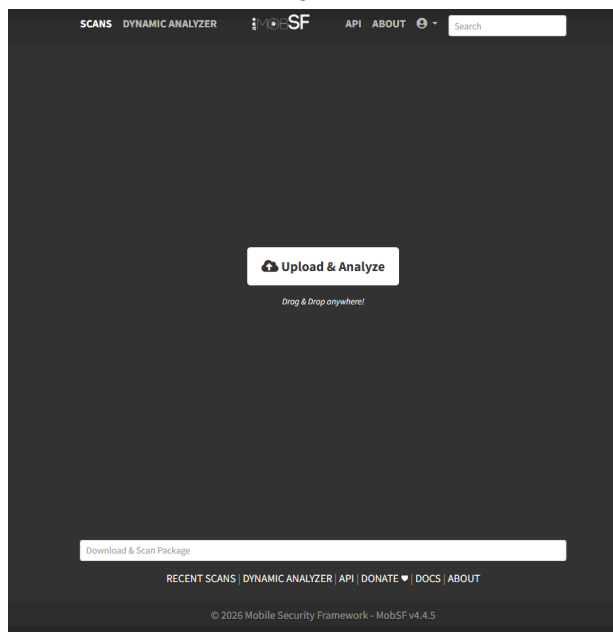
1. Requirements

Docker

- Follow Docker installation here:
<https://docs.docker.com/desktop/setup/install/windows-install/>
- The easiest way is to download **Docker Desktop** and use the command line.
- You can check if installation is complete through typing: **docker** in your terminal.

2. Running MobSF

- Follow the instructions here for: https://mobsf.github.io/docs/#/running_mobsf_docker
- You'll have to run the two commands using the terminal.
- Access the portal at: <http://localhost:8000> or 127.0.0.1:8000
- If required to login use **mobsf** for both email and password.



```

00:01:18 - Analyzing apktool_out/lib/arm64-v8a/libunwindstack
00:01:18 - Analyzing apktool_out/lib/arm64-v8a/libbreakpad_cp
00:01:18 - Analyzing apktool_out/lib/arm64-v8a/libsuperpack-j
00:01:18 - Analyzing lib/arm64-v8a/libfbgloginit.so
00:01:18 - Analyzing lib/arm64-v8a/libsigmux.so
00:01:18 - Analyzing lib/arm64-v8a/libandroid_aware_dlopen.so[INFO]
00:01:18 - Analyzing lib/arm64-v8a/libglog.so
00:01:18 - Analyzing lib/arm64-v8a/libwatcher_binary.so
00:01:18 - Analyzing lib/arm64-v8a/liblinkerutils.so
00:01:18 - Analyzing lib/arm64-v8a/libmemalign16.so
00:01:18 - Analyzing lib/arm64-v8a/libunwindstack_binary.so
00:01:18 - Analyzing lib/arm64-v8a/libbreakpad_cpp_helper.so
00:01:18 - Analyzing lib/arm64-v8a/libsuperpack-jni.so
00:01:18 - Analyzing lib/arm64-v8a/liblmd_detector_binary.so[INFO]
00:01:18 - Analyzing lib/arm64-v8a/libdistract.so
00:01:18 - Analyzing lib/arm64-v8a/libelf.so
00:01:18 - Analyzing lib/arm64-v8a/libfb.so
00:01:18 - Analyzing lib/arm64-v8a/libzstddecoder.so
00:01:18 - Analyzing lib/arm64-v8a/libarc4_sdk_jni.so
00:01:18 - Analyzing lib/arm64-v8a/libforce_dlopen.so
00:01:18 - Reading Code Signing Certificate
00:01:19 - Getting Signature Versions
00:01:26 - Running APKID 3.0.0
00:01:41 - Trackers Database is up-to-date
00:01:41 - Detecting Trackers
00:01:52 - Decompiling APK to Java with JADX
00:06:30 +0000 [1] [INFO] Handling signal: winch
00:06:37 +0000 [1] [INFO] Handling signal: winch

```

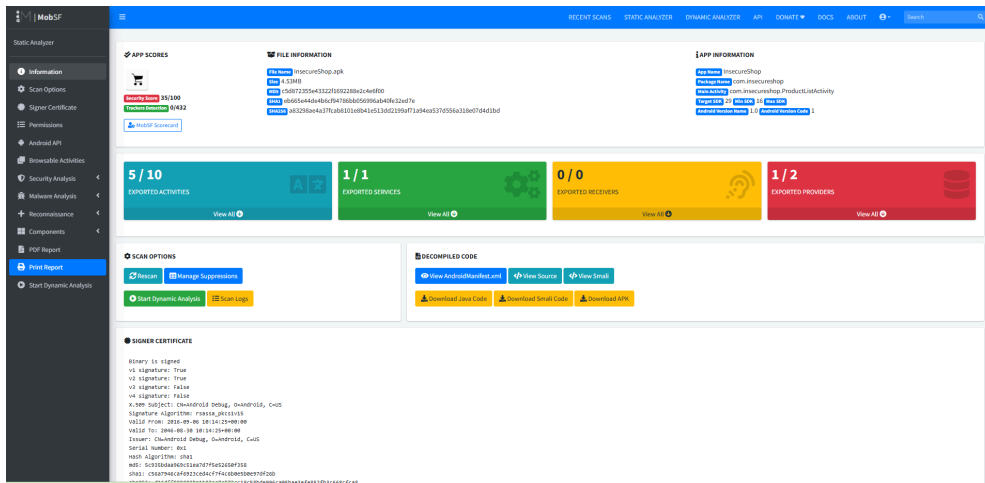
It takes time to run MobSF

```

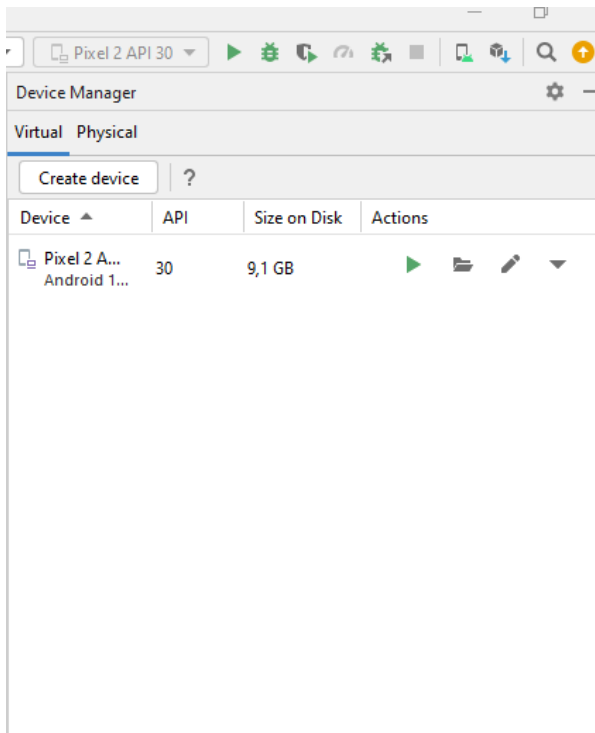
system32
0:45 - Decompiling classes.dex with JADX
0:49 - Decompiling APK to Java with JADX
0:22 - Decompiling with JADX failed for classes.dex
0:27 - Decompiling classes6.dex with JADX
0:41:29 - Decompiling with JADX failed, attempting on all DEX file
0:29 - Decompiling classes.dex with JADX
0:43 - Decompiling with JADX failed for classes6.dex
0:48 - Decompiling classes5.dex with JADX
0:42 - Decompiling with JADX failed for classes.dex
0:46 - Decompiling classes.dex with JADX
0:33 - Decompiling with JADX failed for classes5.dex
0:36 - Decompiling classes3.dex with JADX
0:44 - Decompiling with JADX failed for classes.dex
0:48 - Some DEX files failed to decompile
0:48 - Converting DEX to Smali
0:48 - Converting classes.dex to Smali Code
0:49 - Code Analysis Started on - java_source
0:58 - Reading file contents for SAST
0:59 - Android SBOM Analysis Completed
0:09 - Android SAST Completed
0:09 - Android API Analysis Started
0:19 - Android API Analysis Completed
0:23 - Android Permission Mapping Started
0:26 - Android Permission Mapping Completed
0:27 - Android Behaviour Analysis Started
0:32 - Android Behaviour Analysis Completed
0:33 - Extracting Emails and URLs from Source Code
0:36 - Email and URL Extraction Completed
0:37 - Extracting String data from APK
0:38 - Extracting String data from Code
0:38 - Extracting String values and entropies from Code
0:46 - Starting Firebase Analysis
0:46 - Looking for Firebase URL(s)
0:46 - Analyzing 2 domains
0:50 - Checking for malware database updates
0:02 - Maltrail Database is outdated!
0:06 - Updating Maltrail Database
0:15 - Malware analysis completed: 0 malicious domains found
0:15 - Saving to Database

```

The final generated from MobSF report should look like this



Dynamic Analysis must be run from a virtual device. E.g **Android Studio**:



You are encouraged to do both **static analysis** and **dynamic analysis** for your report.

Scoutsuite

1. Requirements

- Python - download Python from <https://www.python.org/downloads/>
- Python will give you access to **pip** which will help you install dependencies for ScoutSuite.

- Git - download Git from <https://git-scm.com/install/windows>
- Follow the Setup instructions here:
<https://github.com/nccgroup/ScoutSuite/wiki/Setup#via-git>

2. Login to AWS CLI

First Sign Up/Sign for **AWS** here: <https://aws.amazon.com/account/> and click Sign In
Download the **AWS CLI** here:

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

Open the terminal and type **aws login** which should allow you to login to AWS CLI through the browser.

```
Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Student> aws login
No AWS region has been configured. The AWS region is the geographic location of your AWS resources.

If you have used AWS before and already have resources in your account, specify which region you want to use. If
you have not created resources in your account before, you can pick the region closest to you. For more infor
mation, see https://global-infrastructure/latest/regions/aws-regions.html.

You are able to change the region in the CLI at any time with the command "aws configure set aws.region <re
gion>".

AWS Region [us-east-1]:
Attempting to open your default browser.
If the browser does not open, open the following URL:

https://us-east-1.signin.aws.amazon.com/v1/authorize?response_type=code&client_id=arn:aws:iam::625980481664:root
&scope=aws:iam:*&state=648d5414-9019-49e5-b188-98516be25571&code_challenge_method=SHA-256&code_challenge=
F%2F127.0.0.1%3A51882%2F0auth%2Fcallback&code_challenge=00Qd2wFbrBE7J3qwHhskwnGYV5zn418

Updated profile default to use arn:aws:iam::625980481664:root credentials.
PS C:\Users\Student> |
```

Once both **AWS** and **ScoutSuite** are setup, you should be ready to run:

python scout.py aws

This command must be run from the same directory as ScoutSuite where you did **pip install** which will run ScoutSuite against your AWS credentials and account.

If you get this error:

```

ut[10620]      Launching Scout
ut[10620]      Authenticating to cloud provider
ut[10620]      __main__.py L217: Authentication failure: Missing Dependency: Using
es an additional dependency. You will need to pip install "botocore[crt]" before pro

```

Be sure to run **pip install "botocore[crt]"** for authentication

After it's done analysing your cloud environment, a new folder will be created with your results. Open the HTML with the report.

Your report from ScoutSuite should look like this and if you click on findings there should be reasons why that element was flagged

File C:\Users\Student\Downloads\ScoutSuite-master\ScoutSuite-master\scoutsuite-report\aws-625980481664.html

Scout Analytics Compute Containers Database Management Messaging Network Security Storage Filters

Amazon Web Services > 625980481664

Dashboard

Service	Resources	Rules	Findings	Checks
ACM	0	2	0	0
Lambda	2	0	0	0
CloudFormation	0	1	0	0
CloudFront	0	3	0	0
CloudTrail	0	9	17	17
CloudWatch	0	1	0	0
Codebuild	0	0	0	0
Config	0	1	17	17
Directconnect	0	0	0	0
DynamoDB	0	0	0	0
EC2	34	29	85	493
EFS	0	0	0	0
ElastiCache	0	0	0	0
ELB	0	3	0	0
ELBV2	0	5	0	0
EMR	0	0	0	0

You will have to go through the report and analyse its findings as per the PoE requirements.